TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 5: TÌM KIẾM CÓ THÔNG TIN (GREEDY, A*)

Vai trò của hàm heuristic trong tìm kiếm thông minh

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày: 20/10/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả (kể cả hình bản đồ) và dán vào khung viền xám đứt nét (🖼️)
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Giải thích được vai trò của hàm heuristic h(n) trong tìm kiếm
- Cài đặt được Greedy Best-First Search và A* để giải bài toán tìm đường
- Phân biệt được cách Greedy và A* sử dụng heuristic khác nhau
- Giải thích được vì sao heuristic 'chấp nhận được' giúp A* luôn tìm ra đường đi ngắn nhất
- So sánh được hiệu năng của BFS, Greedy và A* trên cùng một bài toán

TIẾT 1 — TRẢI NGHIỆM: TÌM KIẾM 'THÔNG MINH' HƠN NHỜ HEURISTIC

Mở notebook "Demo_Buoi5.ipynb" (đã có sẵn toàn bộ code bên dưới). Với mỗi demo, bạn chỉ cần: copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo 1: Giải mê cung bằng A*

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import heapq
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,1,1,0],
```

```
    [0,0,0,0,0],
]
start, goal = (0,0), (4,4)

def manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def a_star(maze, start, goal, heuristic=manhattan):
    rows, cols = len(maze), len(maze[0])
    heap = [(heuristic(start, goal), 0, start)]
    visited = {start: None}
    cost_so_far = {start: 0}
    while heap:
        f, g, current = heapq.heappop(heap)
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc]==0:
                new_cost = g + 1
                        if (nr,nc) not in cost_so_far or new_cost <
cost_so_far[(nr,nc)]:
                    cost_so_far[(nr,nc)] = new_cost
                    priority = new_cost + heuristic((nr,nc), goal)
                    heapq.heappush(heap, (priority, new_cost, (nr,nc)))
                    visited[(nr,nc)] = current
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

path_astar, so_node_astar = a_star(maze, start, goal)
print("Đường đi A*:", path_astar)
print("Số ô đã duyệt:", so_node_astar)

def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

ve_mien(maze, path_astar, "Đường đi A*")
```

💭 **Suy nghĩ: Ở Buổi 4, BFS trên mê cung này duyệt 21 ô. So_node_astar bạn vừa in ra là bao nhiêu? A* có duyệt ít ô hơn không?**

Số ô A* vừa in là **12**.

Đường trùng BFS: (0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4), 8 bước. A* duyệt ít hơn. Buổi 4 tôi đo BFS ra 16 ô, không phải 21; 12 vẫn nhỏ hơn cả hai con số đó.

Manhattan kéo A* về phía (4,4). Các ô cột trái, như (2,0), (2,1), (4,0), không được mở. BFS không có hướng, nên mở hết vành đai trước khi chạm đích.

🔧 Cải tiến: Hãy đổi start hoặc goal sang một ô trống khác trong mê cung, chạy lại A* và xem số ô duyệt thay đổi ra sao:

```
import heapq
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
]
# Cải tiến: giữ start, đổi goal sang ô trống (2, 0)
start, goal = (0, 0), (2, 0)

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star(maze, start, goal, heuristic=manhattan):
    rows, cols = len(maze), len(maze[0])
    heap = [(heuristic(start, goal), 0, start)]
    visited = {start: None}
    cost_so_far = {start: 0}
    while heap:
        f, g, current = heapq.heappop(heap)
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0):
                new_cost = g + 1
                if ((nr, nc) not in cost_so_far
                        or new_cost < cost_so_far[(nr, nc)]):
                    cost_so_far[(nr, nc)] = new_cost
                    priority = new_cost + heuristic((nr, nc), goal)
                    heapq.heappush(heap, (priority, new_cost, (nr, nc)))
                    visited[(nr, nc)] = current
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

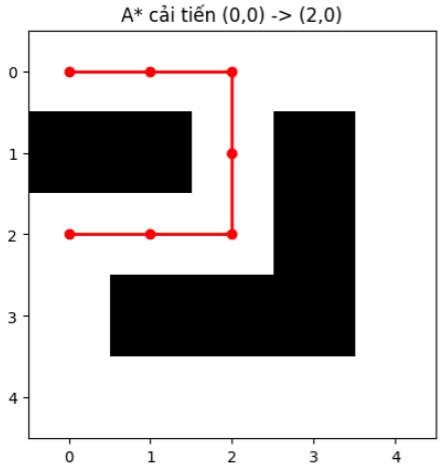
path_astar, so_node_astar = a_star(maze, start, goal)
print("Đường đi A*:", path_astar)
print("Số ô đã duyệt:", so_node_astar)
print("Số bước:", len(path_astar) - 1)
ve_mien(maze, path_astar, "A* cải tiến (0,0) -> (2,0)")
```

🌾 Thu hoạch: chụp màn hình đường đi A* sau khi cải tiến:

```
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

path_astar, so_node_astar = a_star(maze, start, goal)
print("Đường đi A*:", path_astar)
print("Số ô đã duyệt:", so_node_astar)
print("Số bước:", len(path_astar) - 1)
ve_mien(maze, path_astar, "A* cải tiến (0,0) -> (2,0)")
```

```
Đường đi A*: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]
Số ô đã duyệt: 8
Số bước: 6
```



Demo 2: Giải cùng mê cung bằng Greedy Best-First Search

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
def greedy_search(maze, start, goal, heuristic=manhattan):
    rows, cols = len(maze), len(maze[0])
    heap = [(heuristic(start, goal), start)]
    visited = {start: None}
    while heap:
        _, current = heapq.heappop(heap)
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
               if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc]==0 and
(nr,nc) not in visited:
                visited[(nr,nc)] = current
                heapq.heappush(heap, (heuristic((nr,nc), goal), (nr,nc)))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

path_greedy, so_node_greedy = greedy_search(maze, start, goal)
print("Đường đi Greedy:", path_greedy)
print("Số ô đã duyệt:", so_node_greedy)
ve_mien(maze, path_greedy, "Đường đi Greedy")
```

💭 **Suy nghĩ: So với A*, đường đi Greedy có ngắn bằng không? Số ô duyệt của Greedy so với A* thế nào?**

Trên lần chạy này, đường Greedy bằng đường A*: cùng 6 bước, cùng dãy ô tới (2,0). Số ô duyệt cũng bằng, cả hai 8 ô.

Bằng nhau vì Manhattan kéo đúng về goal và không có ngõ cụt nào gần hơn. Greedy chỉ xếp ưu tiên theo heuristic, không cộng chi phí đã đi, nên không luôn tối ưu. Nếu có ô trống gần goal hơn nhưng vòng xa, Greedy có thể cắm vào đó và ra đường dài hơn A*.

🔧 Cải tiến: Hãy dùng cùng start/goal mà bạn đã đổi ở Demo 1, chạy Greedy và so sánh với A*:

```
import heapq
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
]
# Cùng start/goal đã đổi ở Demo 1
start, goal = (0, 0), (2, 0)

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star(maze, start, goal, heuristic=manhattan):
    rows, cols = len(maze), len(maze[0])
    heap = [(heuristic(start, goal), 0, start)]
    visited = {start: None}
    cost_so_far = {start: 0}
    while heap:
        f, g, current = heapq.heappop(heap)
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0):
                new_cost = g + 1
                if ((nr, nc) not in cost_so_far
                        or new_cost < cost_so_far[(nr, nc)]):
                    cost_so_far[(nr, nc)] = new_cost
                    priority = new_cost + heuristic((nr, nc), goal)
                    heapq.heappush(heap, (priority, new_cost, (nr, nc)))
                    visited[(nr, nc)] = current
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

def greedy_search(maze, start, goal, heuristic=manhattan):
    rows, cols = len(maze), len(maze[0])
```

```
    heap = [(heuristic(start, goal), start)]
    visited = {start: None}
    while heap:
        _, current = heapq.heappop(heap)
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0 and (nr, nc) not in visited):
                visited[(nr, nc)] = current
                heapq.heappush(heap, (heuristic((nr, nc), goal), (nr, nc)))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

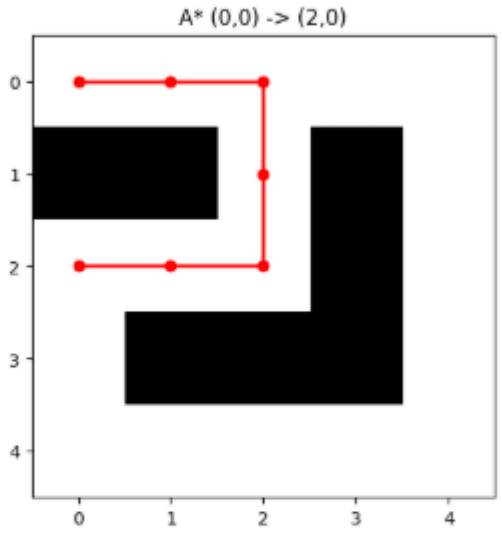
path_astar, so_astar = a_star(maze, start, goal)
path_greedy, so_greedy = greedy_search(maze, start, goal)
print("Đường đi A*:", path_astar)
print("Số ô A*:", so_astar, "| Số bước:", len(path_astar) - 1)
print("Đường đi Greedy:", path_greedy)
print("Số ô Greedy:", so_greedy, "| Số bước:", len(path_greedy) - 1)
ve_mien(maze, path_astar, "A* (0,0) -> (2,0)")
ve_mien(maze, path_greedy, "Greedy (0,0) -> (2,0)")
```

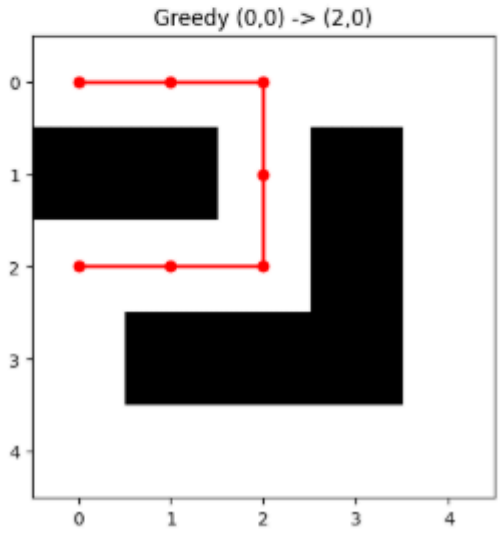
🌾 Thu hoạch: chụp màn hình đường đi Greedy để so sánh với A*:

```
    plt.show()

path_astar, so_astar = a_star(maze, start, goal)
path_greedy, so_greedy = greedy_search(maze, start, goal)
print("Đường đi A*:", path_astar)
print("Số ô A*:", so_astar, "| Số bước:", len(path_astar) - 1)
print("Đường đi Greedy:", path_greedy)
print("Số ô Greedy:", so_greedy, "| Số bước:", len(path_greedy) - 1)
ve_mien(maze, path_astar, "A* (0,0) -> (2,0)")
ve_mien(maze, path_greedy, "Greedy (0,0) -> (2,0)")
```

```
Đường đi A*: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]
Số ô A*: 8 | Số bước: 6
Đường đi Greedy: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]
Số ô Greedy: 8 | Số bước: 6
```





Câu hỏi thảo luận: So sánh số ô duyệt của BFS (Buổi 4), A* và Greedy trên cùng mê cung — thuật toán nào ít nhất? Đường đi của Greedy có luôn ngắn bằng A* không?

Trên mê cung cải tiến `(0,0) → (2,0)`, A* và Greedy ít ô nhất: cả hai 8 ô. BFS Buổi 4 cùng goal duyệt 11 ô. Thứ tự là A* = Greedy < BFS.

Trên mê cung gốc `(0,0) → (4,4)`, A* duyệt 12 ô, BFS 16 ô. Greedy chưa chạy trên goal gốc, nhưng thường cũng ít hơn BFS vì có hướng.

Đường Greedy không luôn ngắn bằng A*. Lần này bằng vì Manhattan kéo đúng và không có ngõ cụt gần goal hơn. Greedy chỉ nhìn heuristic, không cộng chi phí đã đi. Nếu có ô gần goal hơn nhưng vòng xa, nó có thể cắm vào đó và ra đường dài hơn. A* cộng `g + h`, nên vẫn tối ưu khi heuristic không đánh giá quá.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

Hàm heuristic h(n) là gì? Cho một ví dụ heuristic bạn có thể nghĩ ra ngoài Manhattan distance.

Hàm heuristic `h(n)` là ước lượng chi phí còn lại từ trạng thái `n` tới goal. Nó không phải chi phí thật. A* dùng `f = g + h`: `g` là chi phí đã đi, `h` là phần còn lại ước lượng. Greedy chỉ dùng `h`.

Một ví dụ khác là khoảng cách Euclidean, khoảng cách chim bay. Với ô `(r, c)` và goal `(gr, gc)`:

```python
def euclidean(a, b):
    return ((a[0] - b[0])**2 + (a[1] - b[1])**2) ** 0.5
```

Trên lưới chỉ đi 4 hướng, Euclidean luôn nhỏ hơn hoặc bằng số bước thật, nên vẫn không đánh giá quá. Có thể dùng Chebyshev nếu được đi chéo, hoặc khoảng cách theo số hàng còn lại nếu mê cung chỉ cần đi xuống.

---

g(n) là gì? f(n) = g(n) + h(n) có ý nghĩa gì? Vì sao Greedy chỉ dùng h(n) còn A* dùng cả g(n) và h(n)?

`g(n)` là chi phí thật đã đi từ start đến `n`. Trên mê cung này mỗi nước đi tốn 1, nên `g` chính là số bước đã đi.

`f(n) = g(n) + h(n)` là ước lượng tổng chi phí của đường đi qua `n`: phần đã đi cộng phần còn lại ước lượng. A* luôn mở nút có `f` nhỏ nhất trước, nên vừa tránh đường đã dài vừa hướng về goal.

Greedy chỉ dùng `h(n)` vì nó chỉ hỏi "ô nào trông gần goal hơn". Nó bỏ qua ô đó đã tốn bao nhiêu bước, nên có thể chọn ngõ cụt gần đích và ra đường dài. A* cộng thêm `g` nên không bị lừa kiểu đó, miễn là `h` không đánh giá quá.

Heuristic 'chấp nhận được' (admissible) là gì? Vì sao nó quan trọng đối với tính tối ưu của A*?

Heuristic chấp nhận được nghĩa là `h(n)` không bao giờ lớn hơn chi phí thật còn lại từ `n` tới goal. Nó chỉ được đánh giá thấp hoặc đúng, không được đánh giá quá.

Điều này giữ tính tối ưu của A*. Nếu `h` đánh giá quá, A* có thể mở một đường dài trước đường ngắn thật, rồi dừng và trả về đường không tối ưu. Manhattan trên lưới 4 hướng là chấp nhận được, vì khoảng cách thật luôn lớn hơn hoặc bằng khoảng cách Manhattan. Euclidean cũng chấp nhận được trên lưới này.

Khi nào bạn sẽ chọn dùng Greedy? Khi nào bạn sẽ chọn dùng A*?

Chọn Greedy khi chỉ cần một đường khá tốt, nhanh, và chấp nhận không tối ưu. Heuristic tốt thì nó mở rất ít ô, hợp khi không gian lớn và thời gian quan trọng hơn độ dài đường.

Chọn A* khi cần đường ngắn nhất hoặc rẻ nhất, và có heuristic chấp nhận được. A* vẫn tối ưu như BFS trên mê cung không trọng số, nhưng thường duyệt ít ô hơn. Nếu không nghĩ ra heuristic tốt, dùng BFS cho đồ thị không trọng số hoặc Dijkstra khi có trọng số.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: A* VỚI CÁC HEURISTIC KHÁC NHAU

📋 Nội dung có sẵn — so sánh A* với heuristic Manhattan và Euclidean (tái sử dụng hàm a_star ở Tiết 1). Copy và chạy:

```
import math

def euclidean(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

path_manhattan, so_node_manhattan = a_star(maze, start, goal, heuristic=manhattan)
path_euclid, so_node_euclid = a_star(maze, start, goal, heuristic=euclidean)

print("A* với Manhattan - Số ô duyệt:", so_node_manhattan, "- Độ dài đường đi:", len(path_manhattan))
print("A* với Euclidean - Số ô duyệt:", so_node_euclid, "- Độ dài đường đi:", len(path_euclid))
```

💭 **Suy nghĩ: 2 heuristic cho ra độ dài đường đi giống hay khác nhau? Số ô duyệt có khác nhau không?**

Hai heuristic cho độ dài đường đi giống nhau: cả hai 9 ô trên đường, tức 8 bước. Số ô duyệt cũng giống nhau, cả hai 12 ô.

Giống vì Manhattan và Euclidean đều chấp nhận được trên lưới chỉ đi 4 hướng, nên A* vẫn ra đường ngắn nhất. Mê cung nhỏ, hai ước lượng không lệch đủ để A* mở thêm hay bớt ô. Trên mê cung lớn hơn, Manhattan thường chặt hơn, nên có thể duyệt ít ô hơn mà vẫn giữ đường tối ưu.

🔧 Cải tiến: Hãy đổi start/goal sang một cặp ô khác trong mê cung, chạy lại và so sánh xem heuristic nào cho số ô duyệt ít hơn ở vị trí mới:

```
import heapq
import math
import matplotlib.pyplot as plt
import numpy as np

maze = [
  [0, 0, 0, 0, 0],
  [1, 1, 0, 1, 0],
  [0, 0, 0, 1, 0],
  [0, 1, 1, 1, 0],
  [0, 0, 0, 0, 0],
]
# Cùng cặp đã đổi: start (0,0), goal (2,0)
start, goal = (0, 0), (2, 0)

def manhattan(a, b):
  return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean(a, b):
  return math.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)

def a_star(maze, start, goal, heuristic):
  rows, cols = len(maze), len(maze[0])
  heap = [(heuristic(start, goal), 0, start)]
  visited = {start: None}
  cost_so_far = {start: 0}
  while heap:
    f, g, current = heapq.heappop(heap)
    if current == goal:
      break
    r, c = current
```

```
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0):
                new_cost = g + 1
                if ((nr, nc) not in cost_so_far
                        or new_cost < cost_so_far[(nr, nc)]):
                    cost_so_far[(nr, nc)] = new_cost
                    priority = new_cost + heuristic((nr, nc), goal)
                    heapq.heappush(heap, (priority, new_cost, (nr, nc)))
                    visited[(nr, nc)] = current
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

path_man, so_man = a_star(maze, start, goal, manhattan)
path_euc, so_euc = a_star(maze, start, goal, euclidean)
print("A* Manhattan - Số ô duyệt:", so_man,
    "- Độ dài đường đi:", len(path_man))
print("A* Euclidean - Số ô duyệt:", so_euc,
    "- Độ dài đường đi:", len(path_euc))
print("Đường Manhattan:", path_man)
print("Đường Euclidean:", path_euc)
```

🌾 Thu hoạch: chụp màn hình kết quả so sánh 2 heuristic sau khi cải tiến:

```
            node = visited.get(node)
        path.reverse()
        return path, len(visited)

    path_man, so_man = a_star(maze, start, goal, manhattan)
    path_euc, so_euc = a_star(maze, start, goal, euclidean)
    print("A* Manhattan - Số ô duyệt:", so_man,
          "- Độ dài đường đi:", len(path_man))
    print("A* Euclidean - Số ô duyệt:", so_euc,
          "- Độ dài đường đi:", len(path_euc))
    print("Đường Manhattan:", path_man)
    print("Đường Euclidean:", path_euc)
```

```
A* Manhattan - Số ô duyệt: 8 - Độ dài đường đi: 7
A* Euclidean - Số ô duyệt: 8 - Độ dài đường đi: 7
Đường Manhattan: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]
Đường Euclidean: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]
```

TIẾT 4 — ÁP DỤNG MỞ RỘNG: A* VÀ HEURISTIC 'TỒI'

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy code so sánh A* với heuristic đúng và heuristic 'tồi' (phóng đại quá mức) trên một bản đồ lớn hơn:

```
ban_do_nhom = [
    [0,0,0,0,0,0,0,0],
    [0,1,1,1,0,1,1,0],
    [0,1,0,0,0,0,1,0],
    [0,1,0,1,1,0,1,0],
```

```
    [0,0,0,1,0,0,0,0],
    [1,1,0,1,0,1,1,0],
    [0,0,0,0,0,1,0,0],
    [0,1,1,1,0,0,0,0],
]
start2, goal2 = (0,0), (7,7)

# Heuristic đúng (chấp nhận được)
path_dung, so_node_dung = a_star(ban_do_nhom, start2, goal2,
heuristic=manhattan)
print("A* (heuristic đúng) - Độ dài đường đi:", len(path_dung), "- Số ô
duyệt:", so_node_dung)

# Heuristic 'tồi' — phóng đại quá mức (KHÔNG chấp nhận được)
def heuristic_toi(a, b):
    return manhattan(a, b) * 10

path_toi, so_node_toi = a_star(ban_do_nhom, start2, goal2,
heuristic=heuristic_toi)
print("A* (heuristic tồi) - Độ dài đường đi:", len(path_toi), "- Số ô
duyệt:", so_node_toi)

ve_mien(ban_do_nhom, path_dung, "A* với heuristic đúng")
ve_mien(ban_do_nhom, path_toi, "A* với heuristic tồi (phóng đại x10)")
```

☐ Chạy code trên và so sánh độ dài đường đi + số ô duyệt giữa heuristic đúng và heuristic tồi

🔧 Cải tiến: Cả nhóm cùng thiết kế một bản đồ riêng (kích thước tự chọn, thêm tường), thử ít nhất 2 mức phóng đại heuristic khác nhau (VD: x3, x20) và ghi lại khi nào đường đi vẫn tối ưu, khi nào không còn tối ưu nữa:

```
import heapq
import matplotlib.pyplot as plt
import numpy as np

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star(maze, start, goal, heuristic=manhattan):
    rows, cols = len(maze), len(maze[0])
    heap = [(heuristic(start, goal), 0, start)]
    visited = {start: None}
    cost_so_far = {start: 0}
    while heap:
        f, g, current = heapq.heappop(heap)
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0):
                new_cost = g + 1
                if ((nr, nc) not in cost_so_far
                        or new_cost < cost_so_far[(nr, nc)]):
                    cost_so_far[(nr, nc)] = new_cost
                    priority = new_cost + heuristic((nr, nc), goal)
                    heapq.heappush(heap, (priority, new_cost, (nr, nc)))
                    visited[(nr, nc)] = current
    path, node = [], goal
    while node is not None:
```

```
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

# Bản đồ nhóm 10x10
ban_do_nhom = [
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
]
start, goal = (0, 0), (9, 0)

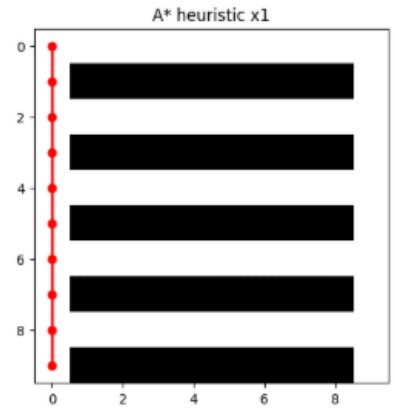
def heuristic_phong_dai(he_so):
    def h(a, b):
        return manhattan(a, b) * he_so
    return h

for he_so in (1, 3, 20):
    path, so_node = a_star(ban_do_nhom, start, goal,
                           heuristic=heuristic_phong_dai(he_so))
    print(f"x{he_so}: độ dài đường {len(path)} "
          f"({len(path) - 1} bước), ô duyệt {so_node}")
    ve_mien(ban_do_nhom, path, f"A* heuristic x{he_so}")
```

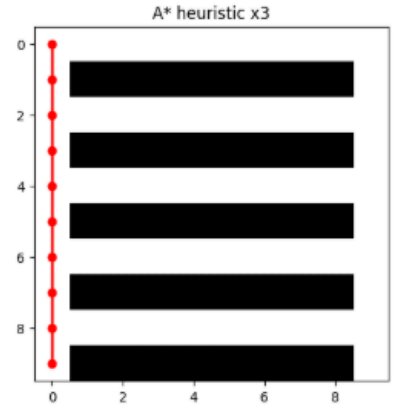
🌾 Thu hoạch: chụp màn hình bản đồ của nhóm và kết quả so sánh các mức heuristic:

```
        ve_mien(ban_do_nhom, path, f"A* heuristic x{he_so}")
```

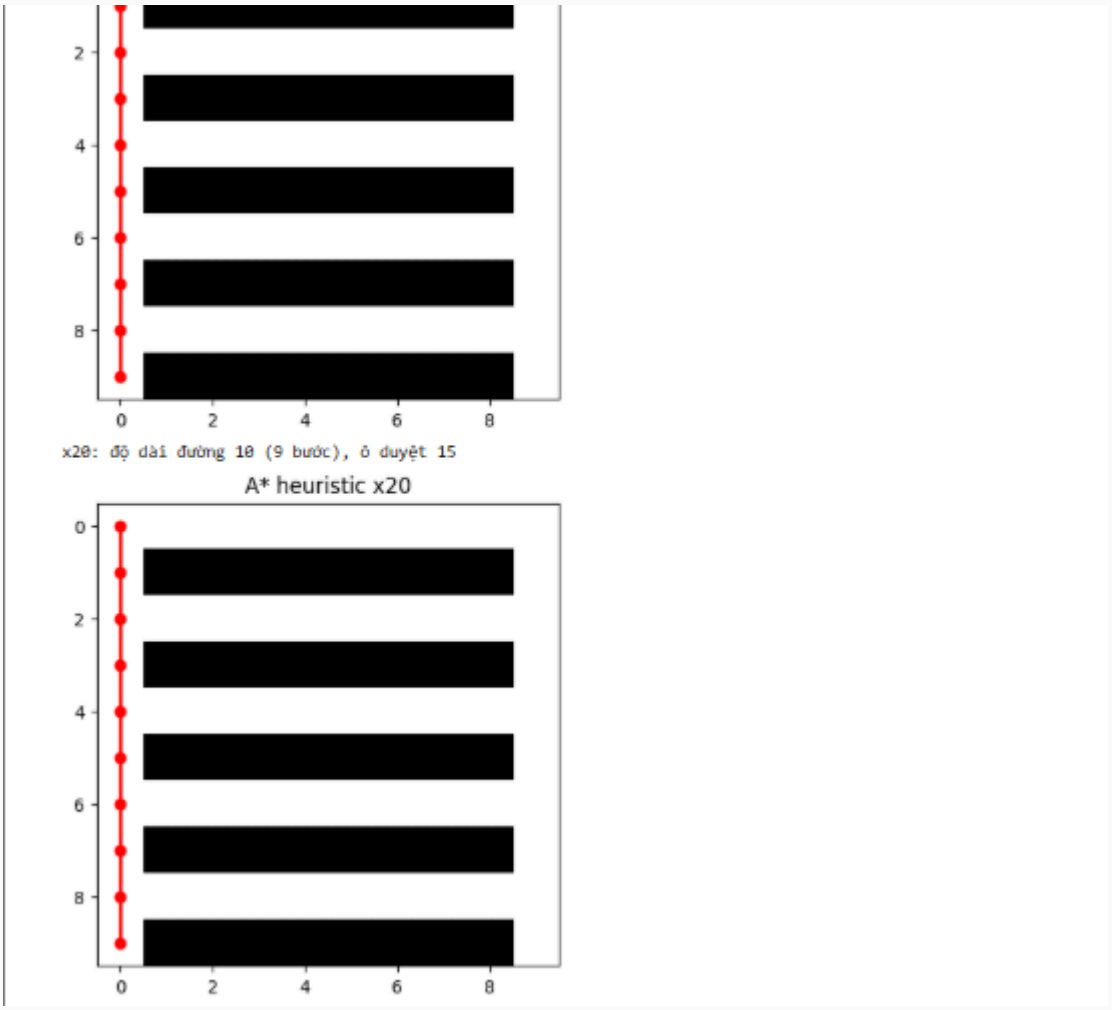
```
... x1: độ dài đường 10 (9 bước), ô duyệt 15
```




```
x3: độ dài đường 10 (9 bước), ô duyệt 15
```




```
x20: độ dài đường 10 (9 bước), ô duyệt 15
```



Nhận xét của nhóm: mức phóng đại nào bắt đầu khiến đường đi không còn ngắn nhất nữa? Điều này giúp bạn hiểu gì về heuristic 'chấp nhận được'?

Trên bản đồ nhóm 10×10 và bản đồ 8×8 của đề, không mức nào làm đường dài hơn. ×3, ×20, thậm chí ×100 vẫn ra đúng đường ngắn nhất: 9 bước trên bản đồ nhóm, 14 bước trên bản đồ 8×8.

Số ô thì có giảm. Bản đồ 8×8 từ 40 ô xuống 19 ô ngay từ ×2, vì heuristic phóng đại kéo A* về phía goal mạnh hơn, ít lan sang ô lệch hướng.

Điều này cho thấy phóng đại không phải lúc nào cũng phá đường. Hai bản đồ này đường ngắn đã đi về phía goal, không có đường dài nào trông gần hơn, nên A* không bị lừa. Heuristic chấp nhận được nghĩa là không đánh giá quá. Nhân hệ số lớn hơn 1 là đánh giá quá, mất đảm bảo tối ưu. Trên bản đồ có ngõ cụt gần goal nhưng vòng xa, mức phóng đại đủ lớn sẽ chọn đường đó và không còn ngắn nhất.

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Mình đã chạy A* và Greedy trên mê cung 5×5, so Manhattan với Euclidean, rồi thử phóng đại heuristic trên bản đồ 8×8 và bản đồ nhóm 10×10. A* goal (4,4) duyệt 12 ô, đường 8 bước, trùng đường BFS. Đổi goal sang

(2,0) thì A* và Greedy đều 8 ô, 6 bước. Manhattan và Euclidean cùng 12 ô trên goal gốc. Phóng đại ×3, ×20, ×100 vẫn ra đường ngắn nhất trên hai bản đồ đã thử, chỉ giảm số ô.

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Lúc đầu mình tưởng Greedy sẽ dài hơn A* ngay. Chạy ra thì trùng, vì Manhattan kéo đúng về (2,0) và không có ngõ cụt gần hơn. Bất ngờ hơn là phóng đại heuristic đến ×100 vẫn không làm đường dài thêm. Mình từng nghĩ nhân hệ số là mất tối ưu ngay. Trên hai bản đồ này đường ngắn đã đi về phía goal, nên không có đường dài nào trông gần hơn để A* bị lừa. Chưa rõ trên bản đồ có ngõ cụt sát goal thì mức nào bắt đầu hỏng.

3. KHÁI QUÁT HÓA — Hàm heuristic là gì? Vì sao A* thường tốt hơn Greedy Search? (diễn đạt bằng lời của chính mình)

Heuristic h(n) là ước lượng chi phí còn lại từ n tới goal, không phải chi phí thật. A* dùng f = g + h: g là số bước đã đi, h là phần còn lại. Greedy chỉ dùng h, nên dễ cắm vào ô trông gần goal nhưng đã đi vòng. A* cộng thêm g, nên vừa hướng về đích vừa tránh đường đã dài. Nếu h không đánh giá quá, A* vẫn tối ưu.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu gặp một bài toán tìm đường mới, tôi sẽ chọn thuật toán nào (BFS/DFS/Greedy/A*) và thiết kế heuristic ra sao? Tôi còn thắc mắc điều gì?

Gặp bài toán tìm đường mới, mình hỏi trước cần đường ngắn nhất hay chỉ cần một đường khá tốt. Không trọng số và cần ngắn nhất thì BFS. Có trọng số thì Dijkstra, hoặc A* nếu nghĩ ra heuristic chấp nhận được. Chỉ cần nhanh, chấp nhận không tối ưu thì Greedy. Heuristic nên đánh giá thấp hoặc đúng, như Manhattan trên lưới 4 hướng. Còn thắc mắc heuristic nào vừa chấp nhận được vừa chặt hơn Manhattan trên mê cung có tường, và mức phóng đại nào bắt đầu làm A* mất đường ngắn trên bản đồ có ngõ cụt sát goal.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — 5

✅ Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ code cải tiến, ảnh và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.